SEGUNDA PARTE — EJERCICIOS (para imprimir y contestar a lápiz)

Nombre: Brayan Usciel Seseña Hidalgo_____**Equipo:** 4_____**Fecha: 24/09/2026**

Sección A0 — Repaso de Python (encierra la letra)

A0.1. En Python, PUERTO = 5050 crea:
a) una función b) una variable de tipo entero c) una librería d) un socket

A0.2. b"hola" es un valor de tipo:

a) str b) int c) bytes d) bool

A0.3. El ciclo while True: termina cuando:
a) llega a 10 vueltas b) se ejecuta un break c) se acaba la memoria d) nunca puede terminar

A0.4. conexion, direccion = servidor.accept() funciona porque accept() devuelve:
a) una lista b) un diccionario c) una tupla de dos elementos d) un bool

A0.5. Un entorno virtual (venv) sirve para:

a) hacer Python más rápido b) aislar las librerías y versiones de cada proyecto c) conectarse a internet d) cifrar el código

Sección A — Opción múltiple (encierra la letra)

A1. Un sistema distribuido se caracteriza porque sus componentes:

a) comparten la misma memoria RAM b) están en computadoras conectadas por red y se comunican solo por paso de mensajes c) corren en la misma máquina d) no necesitan protocolos

A2. En el modelo cliente-servidor:
a) el servidor inicia la comunicación b) el cliente inicia y el servidor espera en una dirección conocida c) ambos inician al mismo tiempo d) no hay direcciones conocidas

A3. ¿Cuántos bytes puede representar un valor de 8 bits (un byte)?
a) 8 b) 127 c) 256 d) 1024

A4. El número ASCII 10 corresponde a:

a) el espacio b) la letra A c) el salto de línea (\n) d) el dígito 0

A5. El puerto bien conocido de HTTPS es:

a) 80 b) 22 c) 443 d) 53

A6. La llamada accept() devuelve:

a) solo la dirección del cliente b) un socket nuevo para hablar con ese cliente y su dirección c) el número de clientes conectados d) el backlog disponible

A7. Los tres mensajes del handshake TCP, en orden:

a) ACK, SYN, SYN-ACK b) SYN, SYN-ACK, ACK c) SYN, ACK, FIN d) HELLO, SYN, ACK

A8. recv(1024) significa:
a) recibe exactamente 1024 bytes b) recibe exactamente un mensaje c) recibe HASTA 1024 bytes de lo que haya llegado d) recibe 1024 mensajes

A9. En UDP, la dirección del remitente se obtiene con:

a) accept() b) connect() c) recvfrom() d) bind()
A10. Para enviar un ARCHIVO de 3 MB de forma confiable, el transporte adecuado es:

a) UDP, porque es más rápido b) TCP, porque garantiza entrega y orden c) da igual d) ninguno, los archivos no viajan por red

A11. Para video EN TIEMPO REAL, el transporte adecuado es:

a) TCP, porque retransmite los cuadros perdidos b) UDP, porque un cuadro perdido retransmitido llegaría tarde c) da igual d) Telnet

A12. El propósito principal de RPC es:
a) cifrar mensajes b) hacer la red más rápida c) transparencia de acceso: llamar lo remoto como si fuera local d) eliminar el sistema operativo

Sección B — Verdadero o falso (encierra y justifica en la línea)

B1. En un sistema distribuido, una parte puede fallar mientras las demás siguen funcionando; tu programa debe contemplarlo. **V** / **F**

B2. La memoria compartida es el modelo natural para comunicar máquinas distintas por red. **V** / **F**

B3. 1 KB equivale a 1000 bytes exactamente. **V** / **F**

B4. El carácter ñ ocupa exactamente 1 byte en UTF-8. **V** / **F**

B5. El socket que devuelve accept() es el mismo que hizo listen(). **V** / **F**

B6. TCP garantiza que cada send() corresponde a exactamente un recv(). **V** / **F**

B7. Si un datagrama UDP se pierde, el protocolo lo retransmite automáticamente. **V** / **F**

B8. UDP es ideal para aplicaciones que no pueden permitir la pérdida de datos. **V** / **F**

Sección C — Relacionar columnas (escribe la letra)

Llamada	Tu respuesta	Descripción
1. socket()	c	a. Asocia el socket a una dirección y puerto locales
2. bind()	a	b. Inicia la conexión hacia el servidor (dispara el handshake)
3. listen()	e	c. Crea el punto final de comunicación
4. accept()	h	d. Envía un datagrama indicando el destinatario cada vez
5. connect()	b	e. Pone el socket en modo escucha
6. recvfrom()	g	f. Cierra el punto final de comunicación
7. sendto()	d	g. Recibe un datagrama y la dirección de quien lo envió
8. close()	f	h. Acepta una conexión y devuelve un socket nuevo + dirección

Sección D — Respuesta corta (usa las líneas)

D1. Define con tus palabras "sistema distribuido" y menciona DOS problemas que no existen en una sola máquina.

Es in sistema donde varios equipos de cómputo se conectan entre si y sus recursos, siendo la red su principal medio de conexión y su comunicación por paso de mensajes.

1. Latencia: ya que los mensajes tardan en viajar

2. Pérdida o desorden: según el transporte que se use algunos mensajes pueden perderse.

D2. Explica la regla de oro "ENVIAR = ENCODE · RECIBIR = DECODE" y qué error aparece si se olvida cada mitad.

Encode consiste en convertir los datos a una representacion de bytes dado una asociacon con una codificacion como utf-8, mientras que decode es el proceso inverso.

D3. ¿Por qué el servidor necesita bind() y el cliente no? ¿Qué hace el sistema operativo por el cliente?

El servidor necesita bind() ya que debe estar en una dirección conocida y fija para ser encontrado o alcanzado por los clientes. Mientras que el cliente no necesita ser buscado, el sistema operativo se asigna un puerto efímero automático.

D4. Explica el problema del framing y describe DOS soluciones.

TCP solo trasporta cierta cantidad de bytes (1024 por lo general) sin dejar en claro donde inicia y termina cada mensaje que puede venir partido o pegado con otros mensajes.

1. Delimitador: definir una marca que dará fin a cada mensaje, por ejemplo el carácter de salto de linea "\n"

2. Prefijo de longitud: se envía primero el tamaño del mensaje, luego se envía el mensaje que es leído por el receptor basado en la cantidad de bytes que se enviaron primero.

D5. ¿Qué son stub y skeleton en RPC, en qué máquina va cada uno, y qué es *marshalling*?

Stub es una función que se encarga de empaquetar del lado del cliente los argumentos y espera por un resultado, mientras que skeleton es la función que recibe y desempaqueta dichos argumentos pero del lado del servidor. Para el proceso de empaquetado del lado del cliente se le conoce como marshalling.

Sección E — Lectura de código

E1. ¿Qué imprime exactamente este programa cuando un cliente se conecta y envía "hola"?

```
import socket
servidor = socket.socket(socket.AF_INET, socket.SOCK_STREAM) servidor.bind(("localhost", 5050))
servidor.listen(1)
conexion, direccion = servidor.accept() datos =
conexion.recv(1024)
print("Me llego:", datos)
print("Tipo:",type(datos))
```

Me llego: b'hola'

Tipo: <Class bytes>

E2. Este código UDP tiene UN error conceptual. Encuéntralo y corrígelo:

```
import socket
servidor = socket.socket(socket.AF_INET, socket.SOCK_DGRAM) servidor.bind(("0.0.0.0",
5050))
servidor.listen(1) se elimina esta línea                    # linea con problema
datos, direccion = servidor.recvfrom(1024)
```

```
import socket
servidor = socket.socket(socket.AF_INET, socket.SOCK_DGRAM) servidor.bind(("0.0.0.0",
5050))
datos, direccion = servidor.recvfrom(1024)
```

E3. ¿Cuál es el error de este cliente TCP?

```
cliente = socket.socket(socket.AF_INET, socket.SOCK_STREAM) cliente.connect(("localhost", 5050))
cliente.send("hola servidor")
```

No se esta especificando la dirección del servidor a la que se va a conectar y lo que se esta enviando hacia el servidor no esta codificado

---

Sección F — Ejercicios prácticos (en tu computadora; entrega capturas)

Cada ejercicio usa los **pasos de ejecución en dos terminales** del código correspondiente (Tema 20). Verifica la salida esperada antes de contestar.

F1. Ciclos de vida (Códigos 1 y 2). Ejecuta el eco TCP. En una copia del código, subraya:
(a) la línea donde el servidor se bloquea esperando cliente; (b) la línea donde se bloquea esperando datos; (c) la línea del cliente que dispara el handshake.

```python
import socket

PUERTO = 5050

servidor = socket.socket(socket.AF_INET, socket.SOCK_STREAM)

servidor.bind(("0.0.0.0", PUERTO))

servidor.listen(1)
print("Servidor escuchando en el puerto", PUERTO)

#a) en esta linea es donde el servidor se bloquea esperando al cliente
conexion, direccion = servidor.accept()
print("Cliente conectado desde", direccion)

while True:
    #b) esta es la linea donde se bloque esperando datos
    datos = conexion.recv(1024)
    if not datos:
        break
    mensaje = datos.decode("utf-8")
    print("Mensaje recibido:", mensaje)
    eco = "ECO>" + mensaje
    conexion.send(eco.encode("utf-8"))

conexion.close()
servidor.close()
print("Servidor eco terminado")
```

```python
import socket

IP_SERVIDOR = "localhost"
PUERTO = 5050

cliente = socket.socket(socket.AF_INET, socket.SOCK_STREAM)

#c) en esta linea es donde se dispoara el handshake
cliente.connect((IP_SERVIDOR, PUERTO))
print("Conectado al servidor eco. Escribe el mensaje o vacio (Enter) para salir")

while True:
    mensaje = input("Mensaje escrito:")
    if mensaje == "":
        break
    cliente.send(mensaje.encode("utf-8"))
    eco = cliente.recv(1024).decode("utf-8")
    print("Respuesta del servidor:", eco)

cliente.close()
print("Conexion cerrada")
```

```
✔ [04:11 ] lonerism@loreta:~/prog_red/fasciculo 10.30.3.58 60% $ python3 eco_TCP_servidor.py
Servidor escuchando en el puerto 5050
Cliente conectado desde ('127.0.0.1', 49608)
Mensaje recibido: hola
Servidor eco terminado
✔ [04:11 ] lonerism@loreta:~/prog_red/fasciculo 10.30.3.58 61% $
```

```
✔ [04:11 ] lonerism@loreta:~/prog_red/fasciculo 10.30.3.58 60% $ python3 eco_TCP_cliente.py
Conectado al servidor eco. Escribe el mensaje o vacio (Enter) para salir
Mensaje escrito:hola
Respuesta del servidor: ECO>hola
Mensaje escrito:
Conexion cerrada
✔ [04:11 ] lonerism@loreta:~/prog_red/fasciculo 10.30.3.58 61% $
```

F2. Ciclo while True + funciones (Código 1). Modifica el servidor de eco: agrega un contador y una función armar_eco(mensaje, numero) que devuelva ECO #N> mensaje. Prueba con tres mensajes.

```python
import socket

def armar_eco(mensaje, numero):
    eco = f"ECO#{numero}> " + mensaje
    conexion.send(eco.encode("utf-8"))


PUERTO = 5050

servidor = socket.socket(socket.AF_INET, socket.SOCK_STREAM)

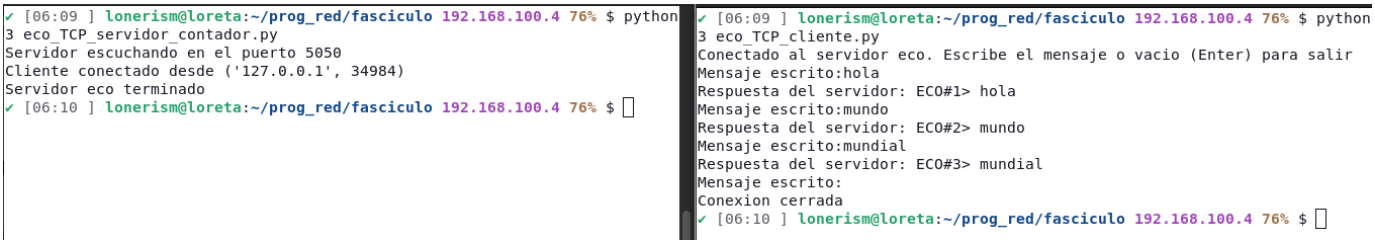
servidor.bind(("0.0.0.0", PUERTO))

servidor.listen(1)
print("Servidor escuchando en el puerto", PUERTO)

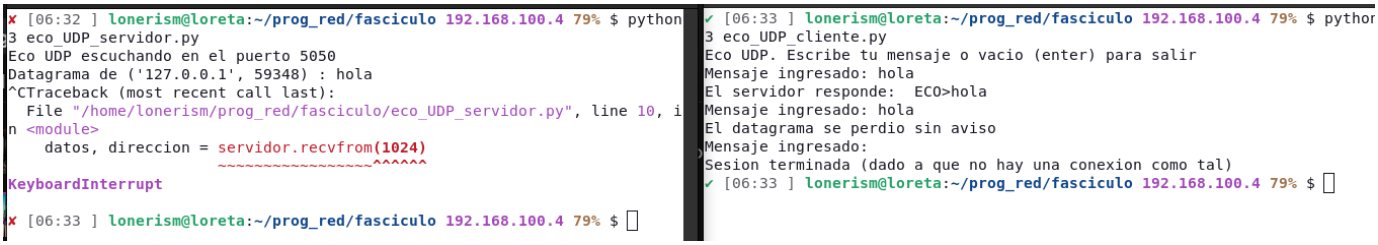
#a) en esta linea es donde el servidor se bloquea esperando al cliente
conexion, direccion = servidor.accept()
print("Cliente conectado desde", direccion)

contador = 1
while True:
    #b) esta es la linea donde se bloque esperando datos
    datos = conexion.recv(1024)
    if not datos:
        break
    mensaje = datos.decode("utf-8")
    armar_eco(mensaje, contador)
    contador += 1

conexion.close()
servidor.close()
print("Servidor eco terminado")
```



F3. UDP y best-effort (Código 4). Ejecuta el experimento de pérdida (apagar el servidor a la mitad). Documenta con captura qué observó el cliente y explica en 3 líneas por qué no hubo ningún error de conexión.



No hubo un erro de conexión ya que el servidor y el cliente nunca establecen del todo una conexión “segura” como si lo hace con TCP, por lo que al apagar el servidor el cliente no experimenta o ve errores, sin embargo, el cliente sigue enviando sus mensajes pero no son recibidos por el servidor.

F4. Framing (Código 5). Ejecuta el servidor y cliente de framing. Luego modifica el cliente para enviar TRES mensajes en un solo send() y verifica que el servidor los separa. ¿Qué línea del servidor es la que "sabe" dónde termina cada mensaje?

```python
import socket

PUERTO = 5061

servidor = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
servidor.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
servidor.bind(("localhost", PUERTO))
servidor.listen(1)
print("Servidor framing escuchando en ", PUERTO)

conexion, direccion = servidor.accept()
print("Servidor framing cliente: ", direccion)

buffer = ""
while True:
    datos = conexion.recv(1024)
    if not datos:
        break
    buffer += datos.decode("utf-8")
    while "\n" in buffer:
        mensaje, buffer = buffer.split("\n", 1)
        print("Servidor framing, mensaje completo recibido:", mensaje)

conexion.close()
servidor.close()
```

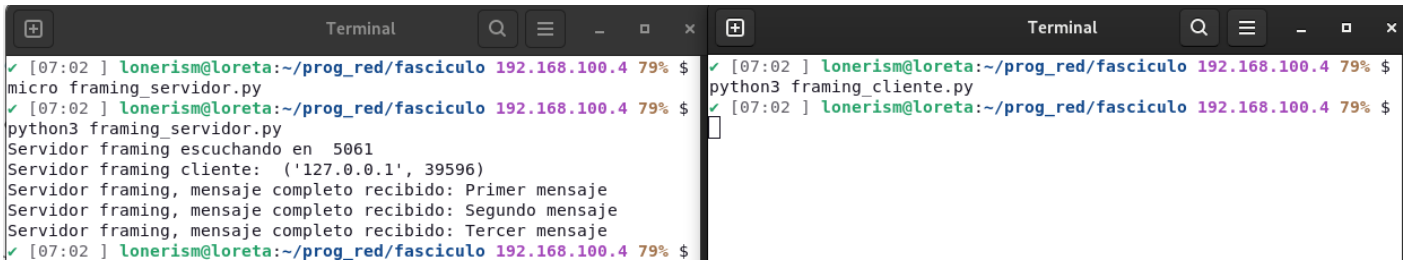
```python
import socket

cliente = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
cliente.connect(("localhost", 5061))

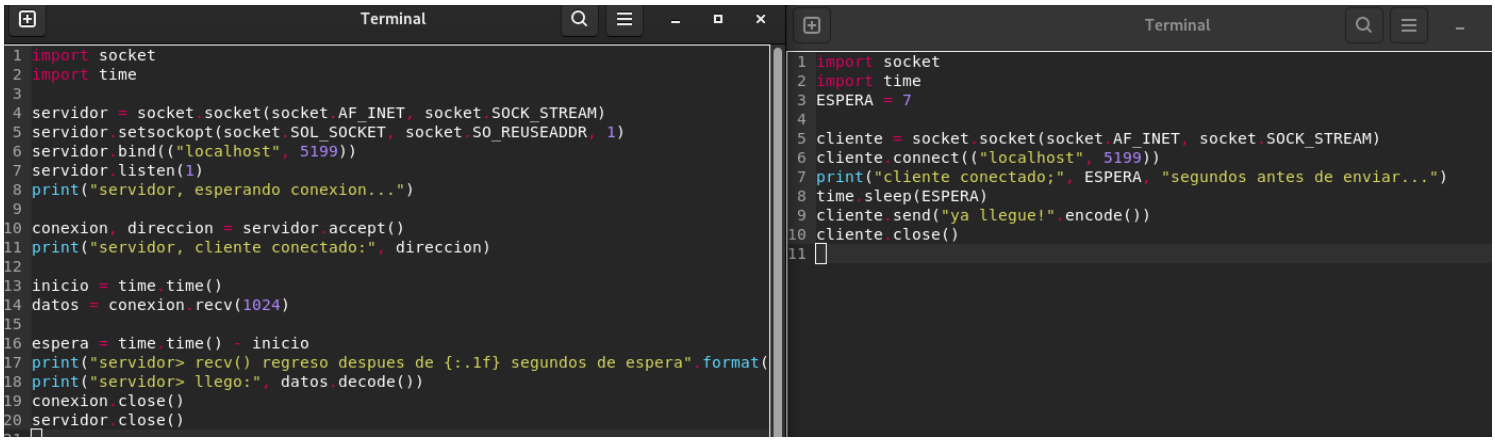
cliente.send("Primer mensaje\nSegundo mensaje\nTercer mensaje\n".encode("utf-8"))
cliente.close()

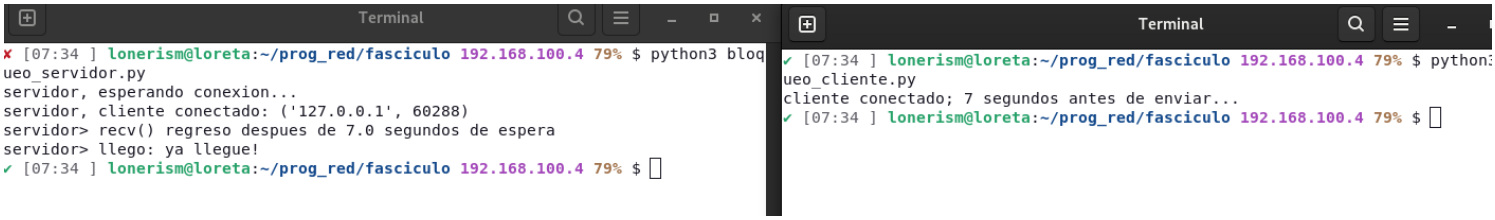

```

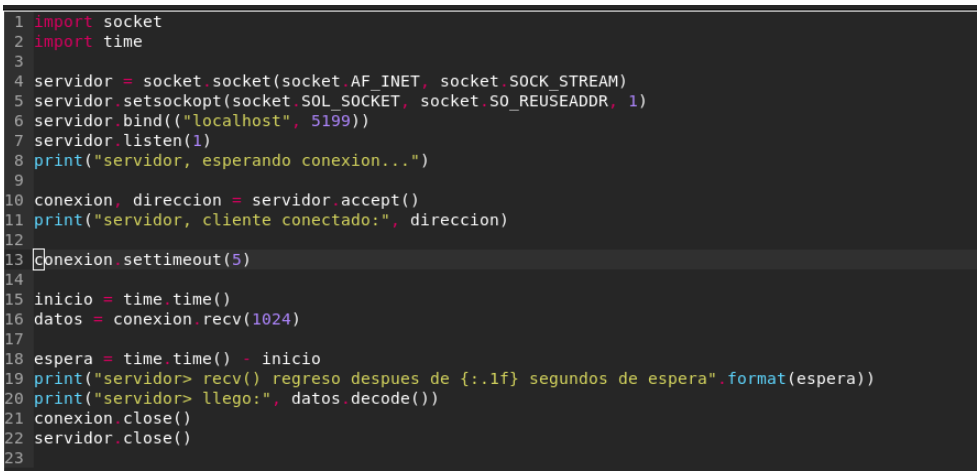
El el caso del servidor la linea en la que sabe en donde termina cada mensaje es en la linea 21, ya que hace uso de la función split(), y usando de delimitador al carácter de salto de linea para dividir cada mensaje.

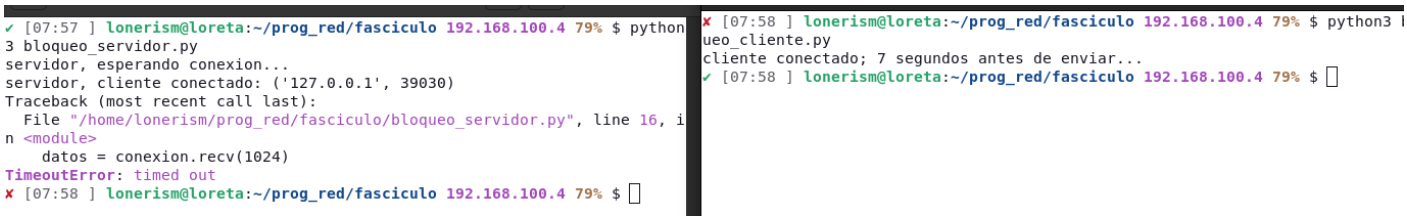


F5. Bloqueo (Código 3). Cambia ESPERA a 7 segundos, ejecuta y anota el tiempo que midió el servidor. Después agrega conexion.settimeout(5) justo después de accept() y vuelve a correr: ¿qué excepción ocurre y en qué lado?









Tras establecer un tiempo de espera en el servidor de 5 segundos, el cliente no logra enviar su mensaje ya que necesita esperar 7 segundos antes de enviarlo, por consecuencia, se genera una excepción del lado del servidor debido al time out establecido.

F6. Código limpio. Reescribe este código cumpliendo TODAS las normas de código limpio (constantes, nombres, comentarios que explican el porqué):

```
s=socket.socket(socket.AF_INET,socket.SOCK_STREAM) s.bind(('',9000))
s.listen(1)
c,a=s.accept()
d=c.recv(1024)
print(d)
```

```python
import socket

PUERTO = 9000
servidor = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
servidor.bind(("localhost", PUERTO))
servidor.listen(1)

conexion, direccin = servidor.accept()
datos = conexion.recv(1024)
print("Datos recibidos: ", datos.decode())

conexion.close()
servidor.close()
```